Ch#02- Intelligent Agents

2.1 Agents and Environments :-

Agent: An agent is anything that can be viewed as :

- perceiving its environment through sensors
- acting upon the environments through actuators.

Example :-

- Human: Eyes, ear, skin | hands, legs, mouth
- Robot: Camera, Infrared sensors, Motors, wheels, robotic arms.
- Software agent: Keyboard input, files, network packets Display files, network messages.

Environment → Sensors → Percepts → Agent → Actions → Environment

.2 Percept :-

A percept is the input an agent receives from its environment through its sensors at a particular moment.

Examples :-

- Camera detects a red light.
- Microphone detects speech
- Touch sensors detects contact.
- Keyboard provides a key press.

Scanned with CamScanner

2.3 Percept Sequence :-

It is the complete history of everything the agent has perceived from the beginning up to the current time.

- Percept Sequence: $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow \ldots\ldots \rightarrow P_n$

An agent's action can depend on its percept sequence, but it cannot directly depend on information it has never perceived.

2.4 Agent Function :-

The agent function is the abstract mathematical description of an agent's behavior.

It maps every possible percept sequence to an action.

$$f : P^* \rightarrow A$$

where:

- P^* = all possible percept sequence
- A = set of possible actions.

Scanned with CamScanner

2.5 Agent Program :-

The agent program is the actual computer program that implements the agent function.

	Agent Function	Agent Program
1.	Abstract description.	Actual implementation
2.	Mathematical concept.	Concrete program
3.	Defines what the agent should do.	Implements how it does it Runs on a machine

2.6 Architecture :-

An architecture is the physical computing system on which the agent program runs. It provides :

- sensors / input to the program
- computational resources
- actuators / output from the program

Therefore : Agent = Architecture + Program.

* The architecture must support the actions the program wants to perform. A program that output walk requires architecture capable of walking.

Scanned with CamScanner

2.7 # Rational Agent :-

A rational agent chooses actions that are expected to maximize its performance measure, given:

1. the percept sequence,
2. the agents prior knowledge,
3. the actions available to it,
4. what it has perceived so far.

- **Definition :-**

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

★ Rationality depends on the <u>situation</u>. The same action may be rational in one environment but irrational in another because the :

- performance measure
- environment
- sensors
- available actions

may be different.

2.8 Performance Measure:-

It is the external standard used to evaluate how successfully an agent performs its task. It tells us: How do we measure whether the agent is doing well?

Example: Vacuum Cleaner

Possible performance measure:

- number of clean squares
- amount of time squares remain clean
- movement cost

2.9 Rationality is not Omniscience:-

An omniscient agent knows the actual outcome of its actions.

Real agents cannot have this ability.

A rational agent chooses the action that is expected to produce the best result based on the information available.

An action can be rational even if an unexpected event later produces a bad result.

Therefore: Rationality ≠ knowing the future

2.10 Rationality and Learning :-

An agent may begin with some prior knowledge supplied by its designer.
However, the environment may contain :

- incomplete information
- unexpected situations
- changes
- incorrect assumptions

A rational agent should therefore be able to learn from experience.

Autonomy :-

An agent is autonomous to the extent that its behavior is determined by its own experience rather than the completely by the prior knowledge supplied by its designer.

A rational agent should become increasingly independent of its initial knowledge through learning.

2.11 Task Environment :-

It is the environment in which an agent performs its task.
The task environment determines :

- what the agent must accomplish
- what it can perceive

- what it can do
- what surrounds it
- how its performance is measured.

The task environment is specified using PEAS.

2.12 PEAS :-

It stands for :

P = Performance measure
E = Environment
A = Actuators
S = Sensors

"PEAS is a description of the task environment for an agent."

When designing an agent, the first step is to specify its task environment as completely as possible.

a) **Performance Measure**: Defines how success is evaluated.
Q: Where ~~does~~ at makes the agent successful?

b) **Environment**: Everything external to the agent that it interacts with :
Q: Where does the agent operate?

Scanned with CamScanner

c) **Actuators**: The mechanism through which the agent performs actions.

Q: How can the agent act?

d) **Sensors**: The mechanisms through which the agent perceives its environment.

Q: How can the agent perceive?

2.13 Properties of Task Environments :-

The nature of the environment strongly affects the design of the agent.

The main properties are:

1. Fully observable vs partially observable
2. Single agent vs multiagent
3. Deterministic vs stochastic
4. Episodic vs sequential
5. Static vs dynamic
6. Discrete vs Continuous
7. Known vs unknown

a) Fully Observable :-

An environment is fully observable when the agent's sensors provide access to the complete state relevant to choosing an action.

- Example: A chess program can see the complete board

b) Partially Observable :-

An environment is partially observable when the agent's sensors do not provide complete information about the current state.

- Example : Taxi cannot see around a large truck.
- Unobservable : If an agent has no sensors, the environment is unobservable.

a) Single-Agent :-

Only 1 agent is being considered as an agent in the environment.

- Example : Crossword puzzle.

b) Multiagent :-

Multiple agents interact in the environment.

- Example : Chess, Soccer, Auction

Competitive & Cooperative Environments :-

a) Competitive : Agents have conflicting performance objectives.
- Example : Chess - One player's success is other's failure.

b) Cooperative : Agents can have compatible performance objectives.

Scanned with CamScanner

- Example: Multiple agents cooperating to avoid collisions.

* Real environments can contain both aspects.

a) **Deterministic :-**

The next state is completely determined by:

- current state
- action taken

- Example: A simple board movement where moving right always produces exactly the same result.

b) **Stochastic :-**

The next state is not completely determined. The same action may have different possible outcomes.

$$\text{Current State} + \text{Action} \longrightarrow \begin{cases} S_1 & P_1 \\ S_2 & P_2 \\ S_3 & P_3 \end{cases}$$

- Example: Driving in traffic. The behavior of other vehicles and unexpected events introduce uncertainty.

Scanned with CamScanner

a) **Episodic :-**

The agent's experience is divided into independent episodes.

- Each episode: Percept → Action

The current action does not effect future episodes.

- Example: Inspecting products on an assembly line: Part 1 → Classify, Part 2 → Classify. The decision about part 1 does not normally affect part 2.

b) **Sequential :-**

Current actions can affect future situations.

- Examples: Chess, Taxi driving.

* A bad action now can create problems later.

a) **Static :-**

The environment does not change while the agent is deciding what to do.

- Example: Chess

b) **Dynamic :-**

The environment can change while the agent is deciding.

- Example: Taxi driving: Other cars continue moving while the agent decides.

Semidynamic :-

The physical environment does not change, but the agent's performance measure changes with time.
Example: Chess with a clock.

a) **Discrete :-**

There are distinct, countable states, actions or time steps.

b) **Continuous :-**

States, actions, or time can take values from a continuous range.

a) **Known :-**

The agent knows:

- what outcomes its actions produce; or
- the probabilities of outcomes if the environment is stochastic.

b) **Unknown :-**

The agent does not know how the environment works and must learn it through experience.

2.14 Agent Structure :-

The basic relationship is:

Agent = Architecture + Program

Scanned with CamScanner

★ Agents Ki types [illegible]

The program receives percepts and produces actions.

The four basic agent program types are:

1. Simple Reflex Agent.
2. Model-Based Reflex Agent.
3. Goal-Based Agent.
4. Utility-Based Agent.

Learning can be incorporated into these agent designs.

2.15 Simple Reflex Agent:-

A simple reflex agent selects an action based only on the current percept. It uses condition-action rules:

IF condition
THEN action

- **Main Advantage**: Simple, fast, easy to implement.
- **Main Limitation**: It cannot properly handle situations where the current percept does not contain enough information.

Therefore, it works best in fully observable environment.

Scanned with CamScanner

2.16 Model-Based Reflex Agent :-

A model-based reflex agent maintains an internal state to deal with the partial observability. It uses:

- current percept
- previous information
- internal state
- model of how the world changes.

- **Model :-**

The model describes :-

1. How the world evolves
2. What the agent's actions do

- **Main Advantage :** Can handle partial observability better than a simple reflex agent.

- **Basic Process :-**

Previous state + Action + Current percept

↓

Update internal state

↓

Match condition-action rule

↓

Action

2.17 Goal-Based Agent :-

A goal-based agent uses goals together with a model of the environment to select actions that will eventually achieve those goals.

Sometimes knowing the current state is not enough to determine what action to take.

The agent also needs to know:

What state does it want to reach?

The information is represented by a goal.

- **Example:** A taxi reaches a junction.
  Possible actions: left, right, straight.
  The correct choice depends on the taxi's destination.
  Therefore, the agent needs:

Current state + Goal + Model of environment

↓

Choose Action

- **Advantage:-**
  More flexible than reflex agents because the same agent can select different actions depending on its goal.

Scanned with CamScanner

2.18 # Utility - Based Agent :-

Goals provide only a basic distinction :

Goal achieved → Good
Goal not achieved → Bad

But sometimes there are many possible outcomes and we need to determine how desirable each one is. This is where utility is used.

- **Utility** : It is an internal measure of how desireable or preferable a state or outcome is to the agent.

A utility function assigns values to states/ outcomes according to their desirability.

- ## Why utility is needed :-

Suppose a taxi wants

- high safety
- short travel time
- low cost

These goals may conflict.

A utility function can represent the appropriate trade - off.

Scanned with CamScanner

Utility is also useful when:

- several goals conflict
- goals cannot all be achieved
- outcomes are uncertain.

2.19 Expected Utility :-

In uncertain environments, an agent may have several possible outcomes.

A rational utility-based agent chooses the action with the highest expected utility.

Conceptually:

$$EU(action) = \sum_i P(outcome_i | action) \times U(outcome_i)$$

where:

- $P(outcome_i | action)$ = probability of the outcome.
- $U(outcome_i)$ = utility of that outcome

Therefore:

Expected utility combines probability and desirability.

The agent chooses the action with the greatest expected utility.

Scanned with CamScanner

2.20 Learning Agent :-

A learning agent improves its behaviour through experience. Its main purpose is :

To modify / improve its behavior based on experience so that future performance becomes better.

A learning agent has 4 main components:

1. Performance element
2. Learning element
3. Critic
4. Problem Generator

1. Performance Element :-

The performance element selects actions. It is the part that actually produces the agent's behavior.

Percepts → Performance element → Actions

2. Learning Element :-

The learning element receives feedback about performance and determines how the performance element should be modified to improve future behavior.

Scanned with CamScanner

Experience → Learning element → Improves performance element.

3. **Critic :-**

The critic evaluates how well the agent is performing against a fixed performance standard. It provides feedback to the learning element.

- The critic is necessary because a percept by itself does not necessarily tell the agent whether its behavior was good or bad.

4. **Problem Generator :-**

The problem generator suggests exploratory actions that may produce useful new experiences.

The agent may temporarily perform an action that is not currently considered optimal in order to learn something that could improve future performance.

- Example : A driving agent may experiment with braking under different road conditions to learn better braking behavior.

★ The learning element can modify different knowledge components of an agent as experience is obtained.

Scanned with CamScanner